

BuoyantCurvatureSwirlTools5 — Algorithm

Rayleigh-discriminant destabilizing-only gate for the curvature/swirl/buoyancy correction | Generation 5 of the BuoyantCurvatureSwirlTools family |

`src/MomentumTransportModels/compressible/BuoyantCurvatureSwirlTools5/`

Purpose

Given the mean velocity field U , its gradient $\text{grad}U$, density ρ , turbulence scales k/ω -like, and eddy viscosity ν_t , produce a per-cell multiplier fr_{Eff} that scales turbulence production G (and $G_b/\text{buoyLimiter}$ for buoyancy production), gated to fire only in regions of the flow that a Rayleigh-discriminant stability criterion identifies as rotationally **destabilizing**. This is the same integration point BuoyantCurvatureSwirlTools3 (BCS3) used — only the diagnostic that decides *where* the curvature correction is allowed to act has changed.

Inputs / Outputs

Inputs

- U (vector field), $\text{grad}U$ (tensor field)
- ρ , k , ω -like, ν_t (scalar fields)
- axisOrigin , axisDirection (rotation-axis geometry)
- coefficients read from the case dictionary (see Coefficients table)

Output — struct `curvatureSwirlData5`

- u_{Theta} , U_{axial}
- L , Φ , F_{gate} (*new in BCS5*)
- fr , fr_{Eff}
- G_b , buoyLimiter

All fields are optionally written to disk when `writeFields = true`.

Step-by-step

STEP 1 Local cylindrical decomposition (*unchanged from BCS3*)

For every cell i , relative to `axisOrigin` / `axisDirection`:

```
rel      = C[i] - axisOrigin
axialProj = rel . axisDirection
vr       = rel - axialProj * axisDirection      (radial component of rel)
r[i]     = |vr|                                (radial distance)
er[i]     = vr / (r[i] + SMALL)                 (radial unit vector)
eTheta[i] = axisDirection x er[i]              (tangential unit vector)
uTheta[i] = U[i] . eTheta[i]
Uaxial[i] = U[i] . axisDirection
```

`radialDistance()` and `radialUnitVector()` are new private helpers — BCS3 only had this loop buried inside `tangentialVelocity()`. They were factored out so `r` and `er` can be reused by the Rayleigh diagnostic below without touching the existing, already-working `tangentialVelocity/axialVelocity` code.

STEP 2 Specific angular momentum and its radial derivative (*new in BCS5*)

```
L      = r . uTheta          [length . velocity]
L^2    = L^2                 [length^4 / time^2]
grad(L^2)          [length^3 / time^2]
dL^2/dr ~= grad(L^2) . er    (directional derivative along the local radial
direction)
r^3     = max(r, rMin)^3      (rMin regularizes the axis singularity)

Phi     = (dL^2/dr) / r^3     [1 / time^2]  <-- the Rayleigh discriminant
```

This is computed as a **local directional derivative**, not a radial binning/averaging step — `grad(L^2)` is the mesh's own cell gradient, dotted with the local radial unit vector. That keeps it a per-cell, unstructured-mesh-friendly operation, consistent with the rest of the file (no new global reduction machinery introduced).

Sign of Φ	Physical meaning
$\Phi > 0$	stabilizing (solid-body-like core)
$\Phi \approx 0$	marginal (free vortex)
$\Phi < 0$	destabilizing (near concave wall)

STEP 3 Normalize Φ into a dimensionless gate (*new in BCS5*)

Φ has dimensions $1/\text{time}^2$ — the same as a strain/rotation-rate squared — so it is normalized against the local mean-flow deformation scale rather than a fixed dimensional constant:

```
D2gate = max(strainMag^2, 0.09 * omegaLike^2)
          (identical formula to the D2 used later in the fr block, Step 4)

Fgate = clamp( -Phi / (cPhi * D2gate + phiSmall), 0, 1 )
```

- The minus sign implements the **destabilizing-only** gate: only $\Phi < 0$ contributes; $\Phi \geq 0$ clamps to 0.
- $c\Phi$ (dimensionless, default 1.0) is a new user-tunable scale factor on the normalization — it plays the role `swirlEps` played in BCS3, but as a multiplicative knob rather than an additive regularizer.
- `phiSmall` = `SMALL` (in Φ 's units) only guards against 0/0 in stagnant regions; it is not a tunable.
- If `swirlCorrection` = `false`, `Fgate` is forced to 0 after being computed (mirrors BCS3's `Fswirl` behavior when its switch is off).

STEP 4 Curvature factor `fr` (*byte-for-byte unchanged from BCS3*)

This entire block — S , Ω , `strainMag`, `rotationMag`, the substantial derivative DS/Dt , $rStar = S/\Omega$, the Spalart–Shur-style `rTilde`, `fRot`, and `fr = max(0, 1 + cCurv * (frTilde - 1))` — is copied verbatim from BCS3. The source equation set's replacement targets only the *gate*, not the curvature-magnitude formula, so this machinery is untouched.

STEP 5 Combine gate and curvature factor

```
if curvatureCorrection:
    frEff = swirlCorrection ? (1 + Fgate*(fr - 1)) : fr
else:
    frEff = 1
```

Identical blend structure to BCS3, with `Fgate` substituted for `Fswirl`.

STEP 6 Buoyancy production (*unchanged from BCS3*)

```
Gb          = Cg * nut * (g . grad(rho)) / rho
buoyLimiter = tanh( |v| / u )
              where v = U . gHat,  u = |U - gHat*v| + uSmall
```

Computed only if `buoyancyCorrection = true` and `gravity` is nonzero. No relation to the swirl/curvature gate.

STEP 7 Write diagnostics

If `writeFields = true` at a write time: `uTheta`, `Uaxial`, `L`, `Phi`, `Fgate`, `fr`, `frEff`, `Gb`, `buoyLimiter` are all written. `L`, `Phi`, and `Fgate` are the three fields genuinely new to BCS5 (BCS3 wrote `Fswirl` in `Fgate`'s slot instead).

Coefficients (dictionary keys)

Key	Default	Role
<code>curvatureCorrection</code> , <code>swirlCorrection</code> , <code>buoyancyCorrection</code>	<code>true, true, false</code>	master switches, same as BCS3
<code>axisOrigin</code> , <code>axisDirection</code>	<code>(0,0,0), (0,0,1)</code>	rotation axis geometry
<code>frMax</code> , <code>cCurv</code> , <code>cr1</code> , <code>cr2</code> , <code>cr3</code>	<code>1.25, 1.0, 1.0, 2.0, 1.0</code>	unchanged, govern <code>fr</code>
<code>rMin</code>	<code>1e-6 m</code>	new — regularizes r^3 near the axis
<code>cPhi</code>	<code>1.0</code>	new — normalization scale on the $\Phi \rightarrow Fgate$ gate
<code>Cg</code>	<code>1.0</code>	unchanged, buoyancy production coefficient
<code>writeFields</code>	<code>false</code>	unchanged

`swirlEps` (BCS3) is gone — Φ 's normalization no longer needs an additive velocity²-scale regularizer.

Design choice made by the implementer: the doc's source equation set left Φ 's normalization/mapping into a bounded gate unspecified. The destabilizing-only mapping `Fgate = clamp(-Phi/(cPhi*D2), 0, 1)` and the choice to normalize against the local deformation scale `D2` (rather than a fixed dimensional constant) were selected in consultation with the user as the most physically direct reading of the Rayleigh-stability sign convention, and to avoid introducing a hard-to-tune dimensional constant.

Not implemented in this generation (flagged as open design choices in the source equation set, no fixed functional form given there):

- Section 4 — circumferential-variance jet-merge applicability gate (σ_θ^2 at fixed r,z).
- Section 5 — near-wall Menter SST F1/F2 attenuation of the gate.

Model family / build integration

BCS5 is wired up exactly like BCS3: a standalone `libbuoyantCurvatureSwirlTools5.so` library, consumed by five RAS model variants that mirror the BCS3-generation family one-for-one, with only the tool class, struct type, and class names renamed:

RAS model	Base turbulence model
kOmegaSST_BCS5	Menter k- ω SST
realizableKE_BCS5_kande	Realizable k- ϵ (frEff applied in both k and ϵ)
kEpsilon_BCS5_kande	Standard k- ϵ
RNGkEpsilon_BCS5_kande	RNG k- ϵ
kOmega_BCS5_kande	Standard k- ω

Build verified via `wmake libso BuoyantCurvatureSwirlTools5` and `wmake libso .` — all five model type names confirmed registered in the resulting `libmycompressibleMomentumTransportModels.so`. No case has been run against BCS5 yet; validation against the jumppipe/Dellenback case is the recommended next step before drawing conclusions.